

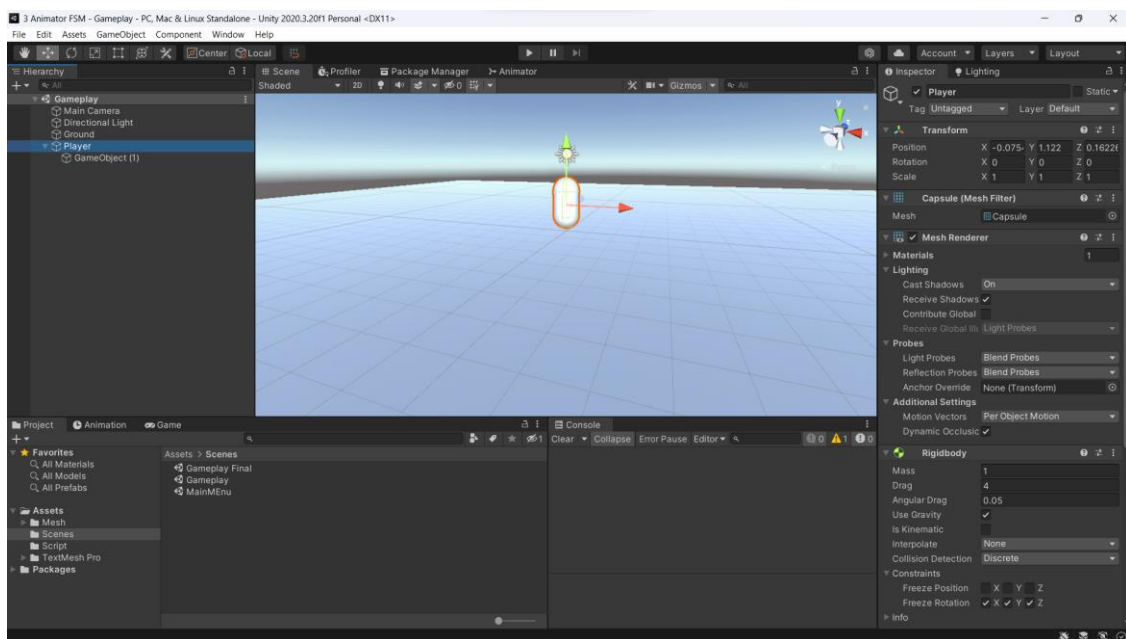
MODUL 3

PRAKTIKUM PEMROGRAMAN MULTIMEDIA & GAME

Animation & Animator FSM

Pada Modul ini, Mahasiswa akan mempelajari tentang memasang animasi dan mengoprasikanya kedalam game. Pada modul ini, praktikan juga akan mempelajari FSM (Finite State Machine) saat mengoprasikan Animator. Tujuan modul ini agar mahasiswa dapat memasang animasi ke karakter, dan juga mempelajari logika FSM untuk relasi antar animasi. Berikut adalah Langkah Langkah praktikumnya.

1. Buka Project Unity Modul 2 Anda



2. Download Resource

Download file karakter dan animasinya di folder modul 3 :

<https://bit.ly/modulpraktikumgame>

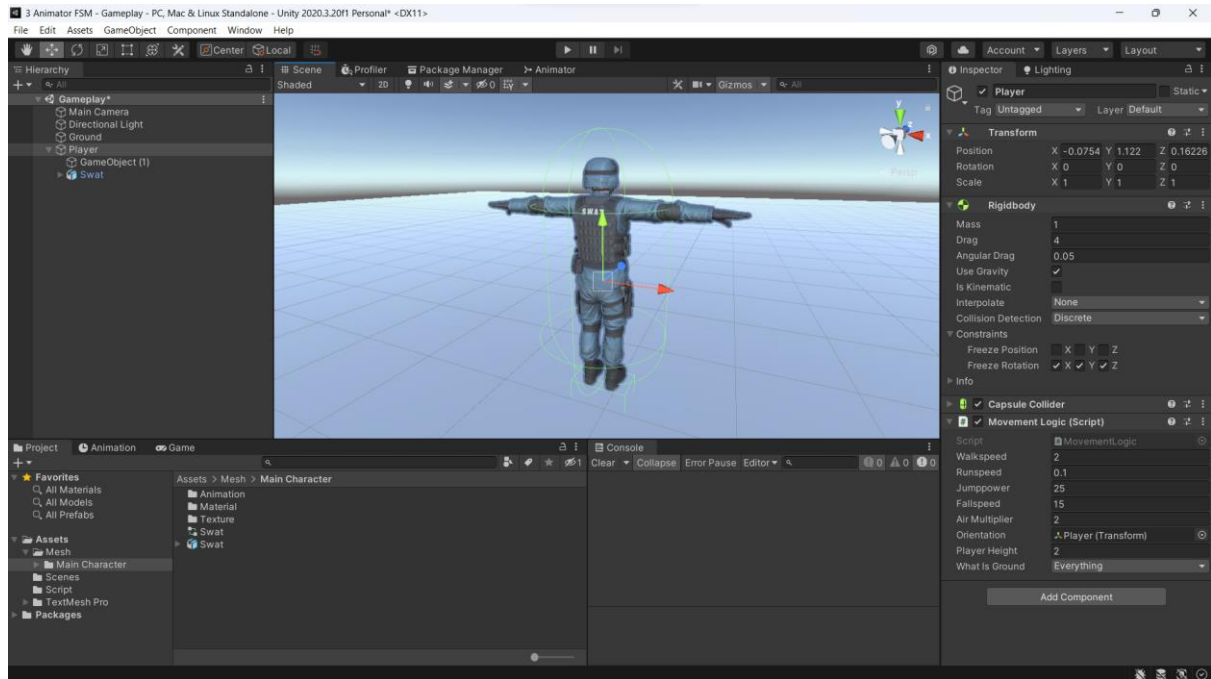
Main Character.unitypackage

Jika sudah, drag and drop ke project unity anda, dan import package.

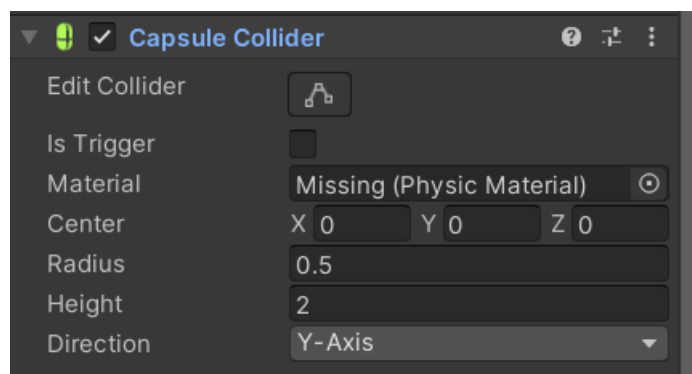
3. Import Asset

Sebelum Memasukan Asset ke Scene, Seleksi GameObject Player, lalu Remove Component Capsule (mesh Filter) dan Component Mesh Renderer.

Setelah itu Masukan Asset **Swat** kedalam Game Object Player.



Atur Posisi Karakter agar pas dengan capsule collider, atau bisa atur Capsule Collider agar memiliki Skala yang tepat.



4. Review Script Modul 2 Sebelum Melanjutkan Modul 3

Cek Kembali script yang ditulis di modul 2, pastikan sudah menyelesaikan materi praktikum dan tugasnya, karena Modul 3 sangat berhubungan dengan Modul 2. Untuk memastikan lagi, berikut adalah script yang digunakan di Modul 2. jika sudah menyelesaikannya, lanjutkan ke step selanjutnya.

```

@ Unity Script (3 asset references) | 2 references
public class MovementLogic : MonoBehaviour
{
    private Rigidbody rb;
    public float walkspeed, runspeed, jumppower, fallspeed, airMultiplier;
    public Transform PlayerOrientation;
    float horizontalInput;
    float verticalInput;
    Vector3 moveDirection;
    bool grounded = true, aerialboost = true;

    @ Unity Message | 0 references
    void Start()
    {
        rb = this.GetComponent<Rigidbody>();
    }
}

```

Unity Message | 0 references

```
void Update()
{
    Movement();
    Jump();
}
```

1 reference

```
private void Jump()
{
    if (Input.GetKeyDown(KeyCode.Space) && grounded)
    {
        rb.velocity = new Vector3(rb.velocity.x, 0f, rb.velocity.z);
        rb.AddForce(transform.up * jumppower, ForceMode.Impulse);
        grounded = false;
    }
    else if (!grounded)
    {
        rb.AddForce(Vector3.down * fallspeed * rb.mass, ForceMode.Force);
        if (aerialboost)
        {
            rb.AddForce(moveDirection.normalized * walkspeed * 10f * airMultiplier, ForceMode.Impulse);
            aerialboost = false;
        }
    }
}
```

1 reference

```
public void groundedchanger()
{
    grounded = true;
    aerialboost = true;
}
```

1 reference

```
private void Movement()
{
    horizontalInput = Input.GetAxisRaw("Horizontal");
    verticalInput = Input.GetAxisRaw("Vertical");
    moveDirection = PlayerOrientation.forward * verticalInput + PlayerOrientation.right * horizontalInput;

    if (grounded)
    {
        if (Input.GetKey(KeyCode.LeftShift))
        {
            rb.AddForce(moveDirection.normalized * runspeed * 10f, ForceMode.Force);
        }
        else
        {
            rb.AddForce(moveDirection.normalized * walkspeed * 10f, ForceMode.Force);
        }
    }
}
```

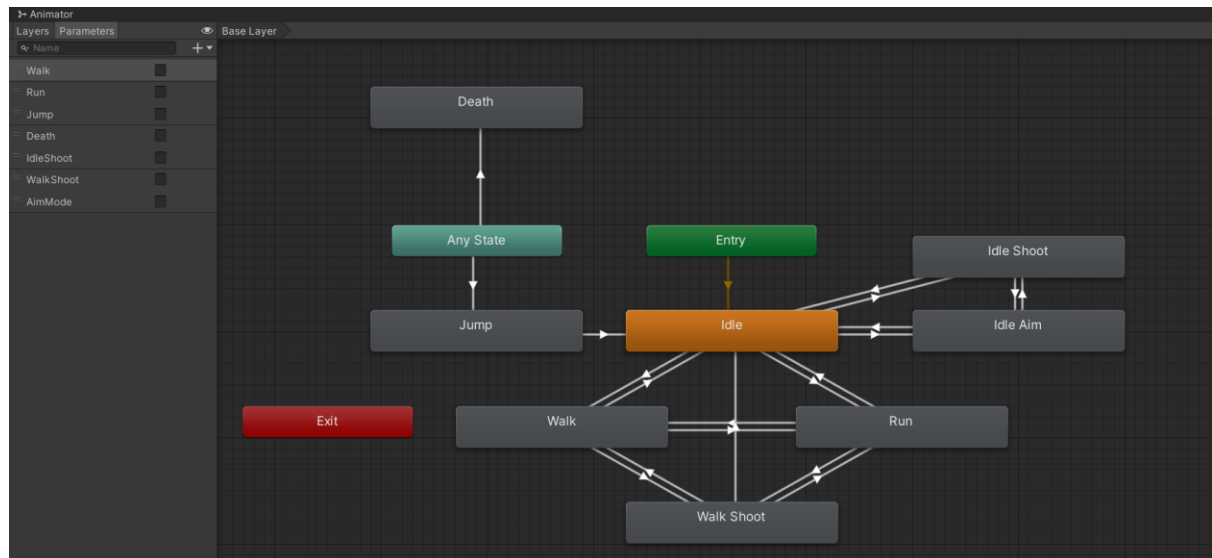
5. Finite-state machine (FSM)

Finite State Machine merupakan salah satu metodologi perancangan sistem kontrol yang menggambarkan tingkah laku atau prinsip sistem dengan menggunakan beberapa element sebagai berikut :

- **Keadaan (state):** Representasi dari kondisi sistem pada titik waktu tertentu.
- **Transisi (transition):** Aturan yang menggambarkan bagaimana sistem berpindah dari satu keadaan ke keadaan lain berdasarkan input tertentu.
- **Input:** Data atau sinyal yang digunakan untuk mempengaruhi transisi antara keadaan.

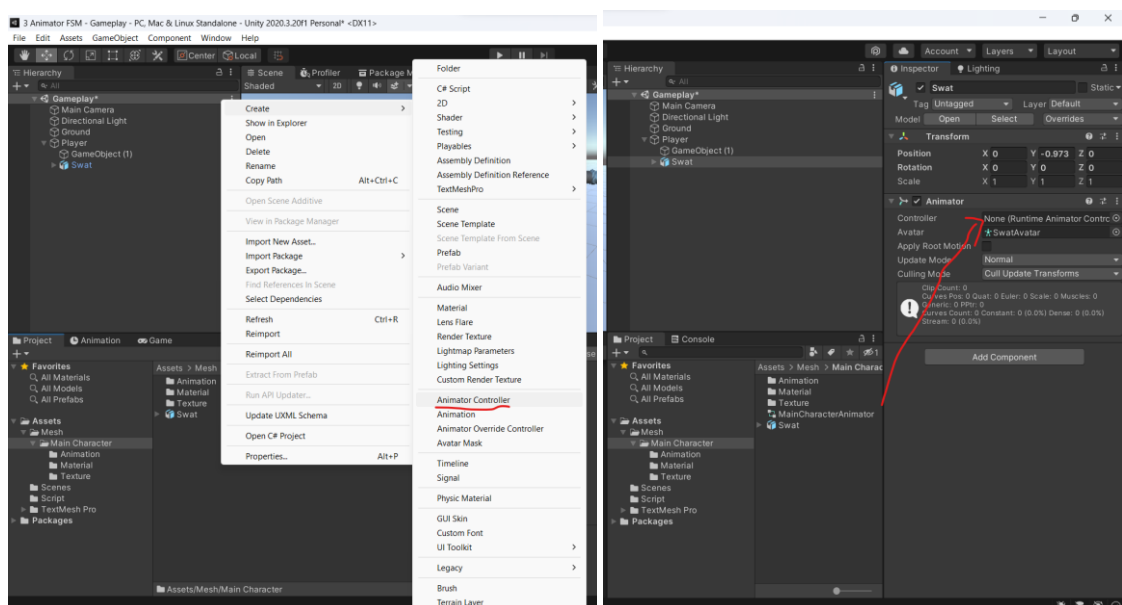
- **Output:** Hasil yang dihasilkan oleh FSM sebagai respons terhadap input dan keadaan saat ini.

Implementasi FSM pada unity engine umumnya digunakan untuk menentukan State animasi yang akan dimainkan, dan juga transisinya. Gambar di bawah ini adalah contoh implementasi pada unity.



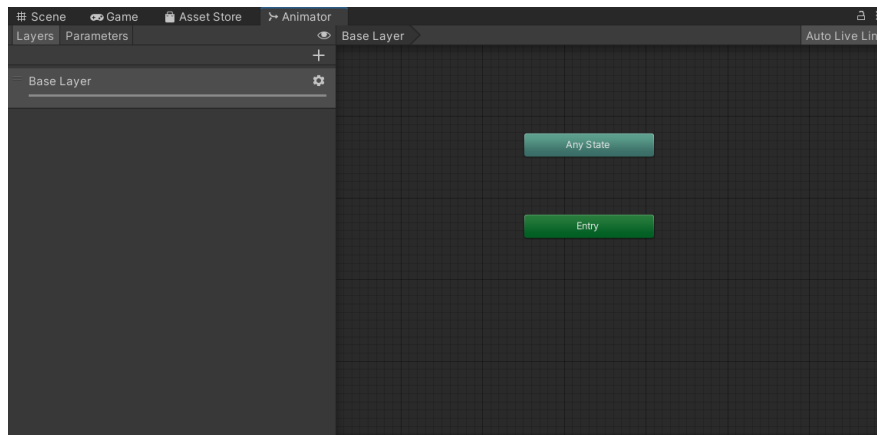
Langkah pertama yang harus dilakukan adalah membuat Animator Controller untuk menampung logika FSM yang akan dibuat.

Project – RightClick – Create – Animator Creator – Drag ke Animator Player



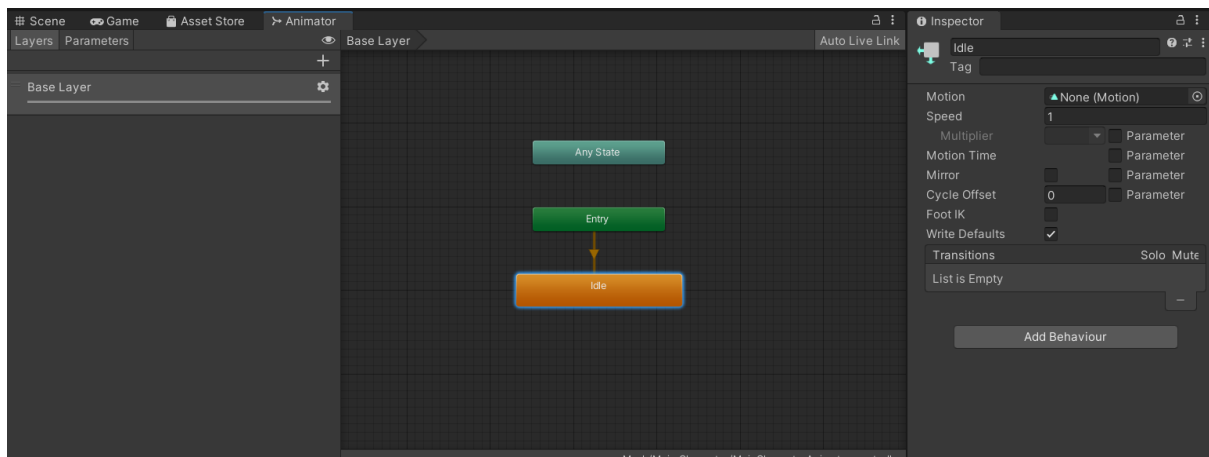
6. Membuat State Awal animasi Idle

Double Click Animator Controller yang telah dibuat, dan akan terlihat tampilan seperti dibawah.

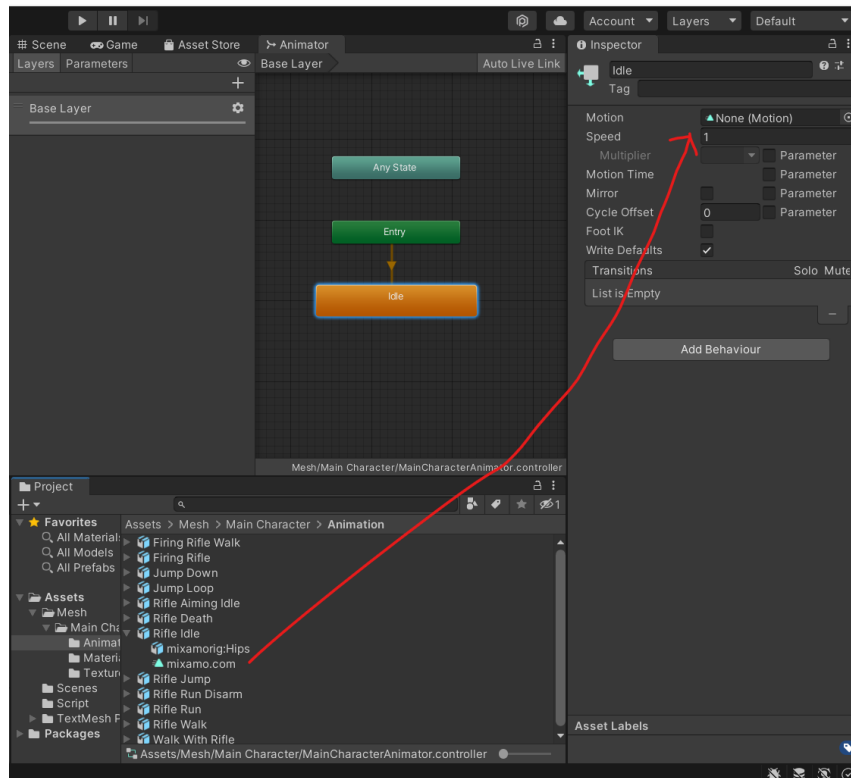


Lanjutkan dengan membuat Logika Animasi Idle, animasi Idle adalah state dimana karakter tidak melakukan aktivitas atau menganggur. Untuk membuat state baru, langkahnya adalah sebagai berikut.

RightClick – Create State – Empty – Rename State Idle



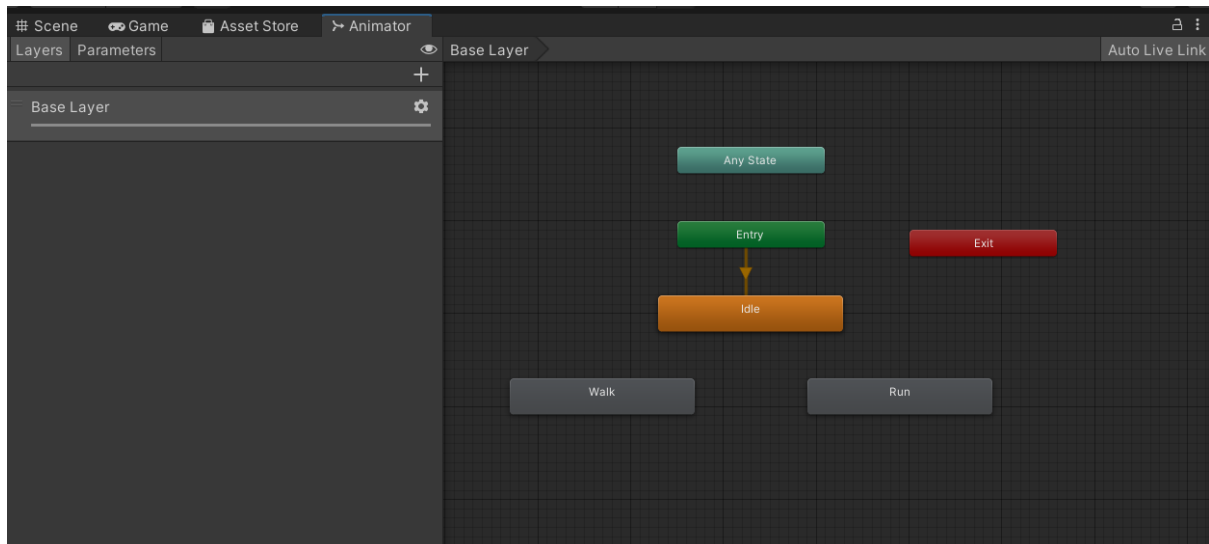
Selanjutnya adalah memasang Motion/Animasi Idle kedalam State Idle. Langkahnya adalah Drag and Drop Animasi Idle di file yang Swat yang telah didownload sebelumnya.



Setelah Memasang Animasi tersebut, Save Scene dan masuk ke **Play Mode**, jika karakter menjalankan animasi idle, maka konfigurasi animasi idle sudah benar. jika masih belum, lakukan pengecekan ulang!

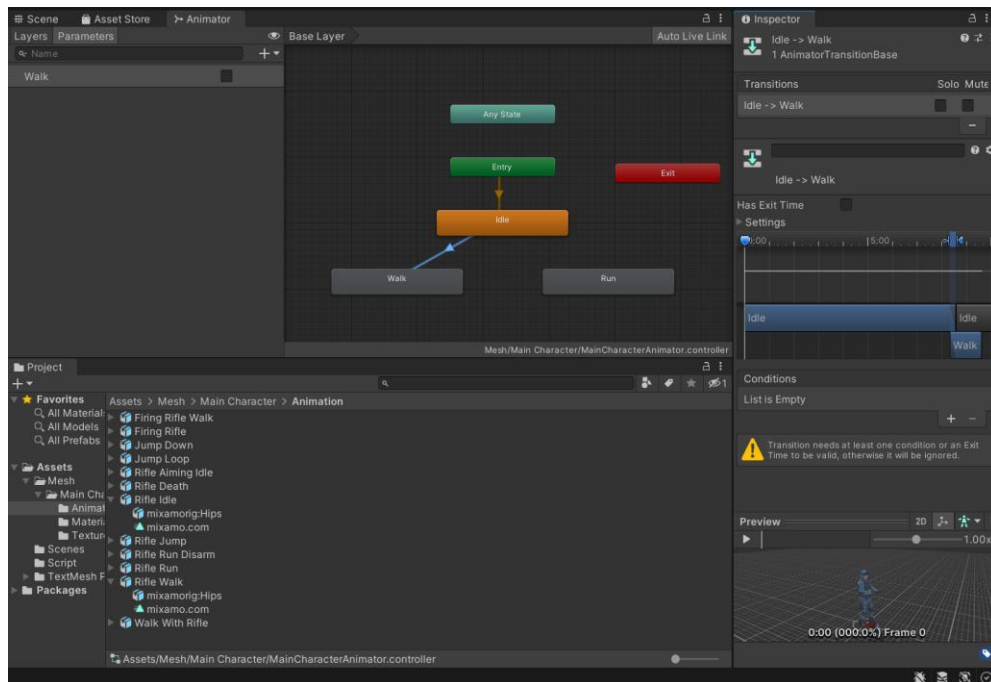
7. Membuat State Walk dan Run.

Sama seperti Langkah sebelumnya, pada animator buatlah State baru dengan nama Walk dan Run. Jika sudah tampilan Animator akan terlihat seperti dibawah ini.



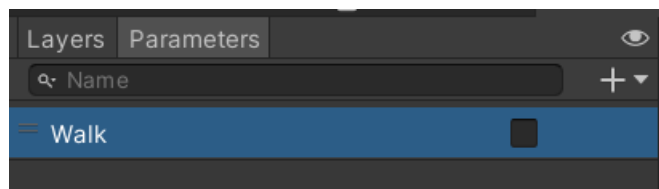
Lanjutkan dengan membuat **Transition** pada setiap **State**, ini berfungsi agar setiap state dapat melakukan transisi animasi. Pertama buatlah transisi dari Idle ke Walk dengan cara

Right Click State Idle – Make Transition – Drag dan Click ke Walk

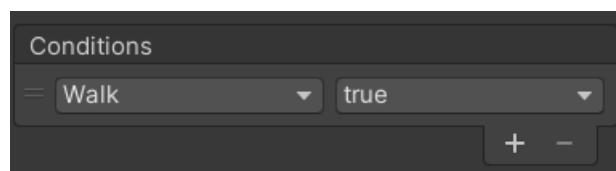


Jika Langkah Benar, akan muncul sebuah panah penghubung yang akan berfungsi sebagai transisi animasi Idle ke animasi Walk, jika dilakukan seleksi pada panah penghubung, akan terlihat sebuah kolom **Condition** dibagian Inspector. Ini menunjukkan jika transisi antar animasi membutuhkan sebuah **Parameter (Input)** yang akan memindahkan State awal ke State Tujuan. Cara membuat **Parameter** baru adalah sebagai berikut

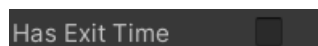
Click Tab Parameter – Click Icon (+) – pilih Bool – Rename Menjadi Walk



Setelah membuat Parameter **Bool Walk**, pasang Parameter tersebut kedalam Transition antara Idle dan Walk. Dengan cara seleksi transisi lalu dibagian inspector click icon (+) dikolom Condition.



Selanjutnya adalah uncheck **Has Exit Time**, hal ini akan membuat transisi animasi idle ke walk dilakukan secara langsung ketika kondisi terpenuhi (**tidak perlu menunggu frame animasi selesai**). Sedangkan jika **Has Exit Time** masih di centang, ini menyebabkan **transisi baru akan dilakukan ketika frame animasi awal sudah selesai di play secara penuh**.



8. Pemrograman Kondisi Idle – Walk.

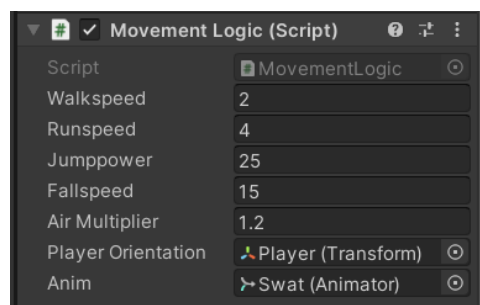
Langkah selanjutnya adalah memprogram Logika agar dapat merubah value boolean menjadi true ketika karakter sedang berjalan. Langkah pertama adalah menambahkan variable global baru seperti dibawah.

```
private Rigidbody rb;
public float walkspeed, runspeed, jumppower, fallspeed, airMultiplier;
public Transform PlayerOrientation;
float horizontalInput;
float verticalInput;
Vector3 moveDirection;
bool grounded = true, aerialboost = true;
public Animator anim;
```

Dan lanjutkan dengan penambahan 1 baris kode didalam kondisi karakter berjalan.

```
if (grounded && moveDirection != Vector3.zero)
{
    if (Input.GetKey(KeyCode.LeftShift) )
    {
        rb.AddForce(moveDirection.normalized * runspeed * 10f, ForceMode.Force);
    }
    else
    {
        anim.SetBool("Walk", true);
        rb.AddForce(moveDirection.normalized * walkspeed * 10f, ForceMode.Force);
    }
}
```

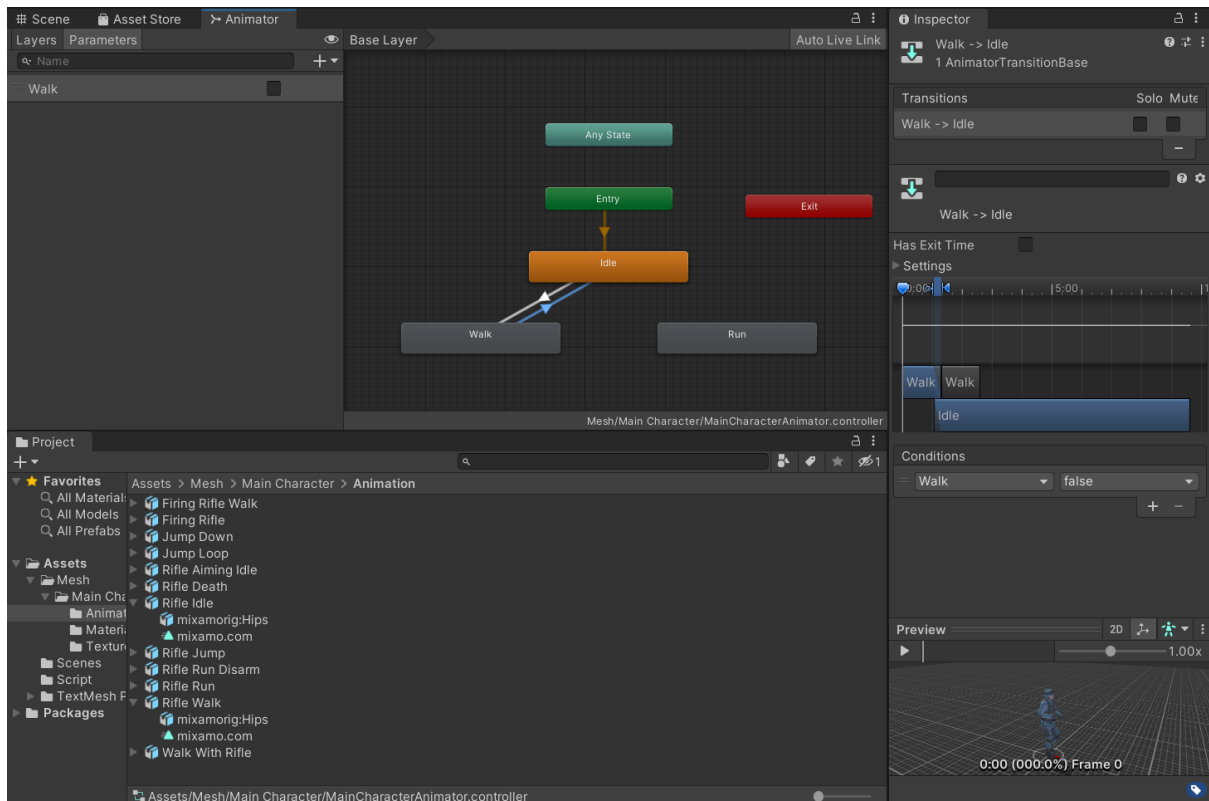
Setelah menuliskan baris kode tersebut, save script dan kembali ke unity. Script Movement Logic akan membutuhkan Value Animator, Drag and Drop **Swat** kedalam Dropdown tersebut.



Setelah itu masuk kedalam **Play Mode**. Lakukan percobaan berjalan, jika karakter memainkan animasi berjalan ketika dilakukan input berjalan, maka script dan konfigurasi sudah benar, jika belum, lakukan pengecekan ulang!

9. Membuat Transisi Walk – idle.

Selanjutnya adalah membuat kondisi ketika karakter tidak berjalan, maka animasi kembali ke idle. Langkahnya adalah pembuatan transisi baru dan setting Condition Transisi (if Walk = False).



Selanjutnya adalah melakukan pemrograman kondisi ketika Karakter tidak berjalan, maka **Walk = False**.

```
1 reference
private void Movement()
{
    horizontalInput = Input.GetAxisRaw("Horizontal");
    verticalInput = Input.GetAxisRaw("Vertical");
    moveDirection = PlayerOrientation.forward * verticalInput + PlayerOrientation.right * horizontalInput;

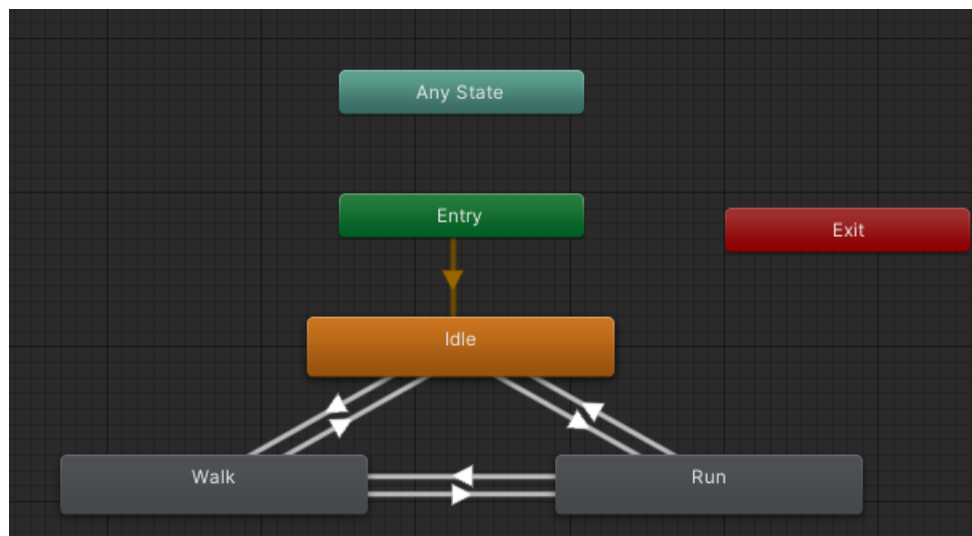
    if (grounded && moveDirection != Vector3.zero)
    {
        if (Input.GetKey(KeyCode.LeftShift))
        {
            rb.AddForce(moveDirection.normalized * runspeed * 10f, ForceMode.Force);
        }
        else
        {
            anim.SetBool("Walk", true);
            rb.AddForce(moveDirection.normalized * walkspeed * 10f, ForceMode.Force);
        }
    }
    else
    {
        anim.SetBool("Walk", false);
    }
}
```

Jika sudah, save script kembali ke unity dan masuk ke **Play Mode**. Jika karakter memainkan animasi berjalan ketika berjalan, dan kembali ke animasi idle ketika tidak berjalan, maka script dan konfigurasi anda sudah benar, jika belum, lakukan pengecekan ulang!

10. Membuat Transisi dan Memprogram logika Idle, Walk, dan Run.

Sama seperti langkah sebelumnya, yaitu pada langkah ini akan membuat transisi Idle ke Run, Run ke Idle, Walk ke Run, dan Run ke Walk. Langkahnya adalah sebagai berikut.

1. Buat **Parameter** Bool Baru Bernama Run.
2. Buat **Transisi** dari Idle ke Run dengan Kondisi **Run = True**
3. Buat **Transisi** dari Run ke Idle dengan Kondisi **Run = False & Walk = False**
4. Buat **Transisi** Walk ke Run dengan Kondisi **Run = True**
5. Buat **Transisi** Run ke Walk dengan Kondisi **Walk = True**
6. Tambahkan **Kondisi** Walk ke Idle dengan kondisi **Run = False**
7. Pasang **Motion/Animasi** State Run dari file "Rifle Run Disarm".



Setelah membuat transisi tersebut langkah selanjutnya adalah memprogram logika transisi tersebut didalam script. Lakukan penambahan script seperti dibawah ini.

```

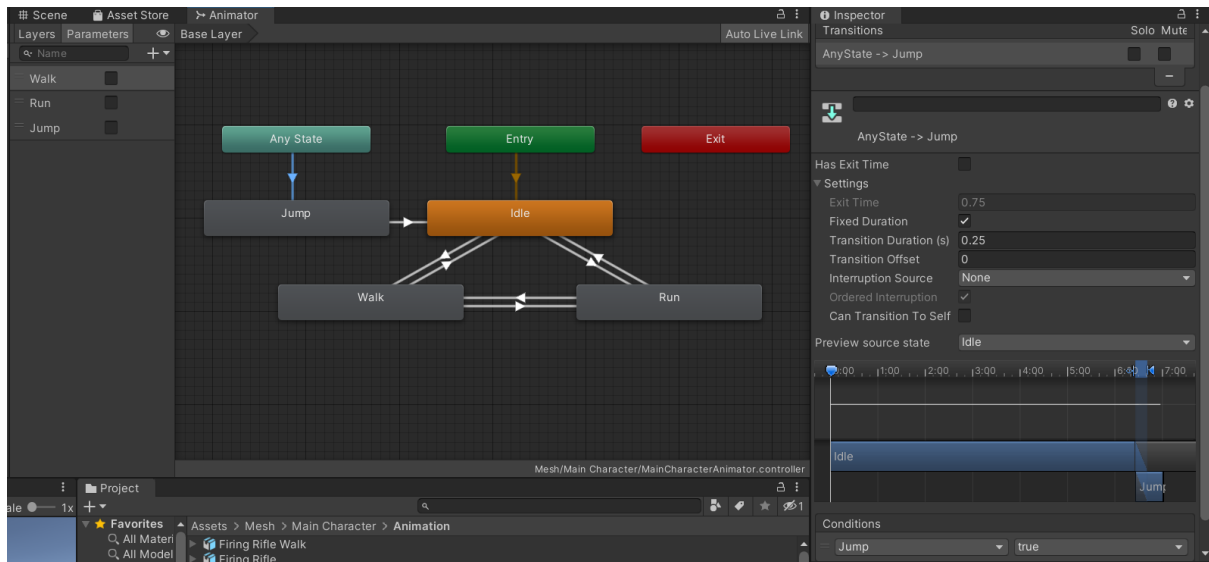
private void Movement()
{
    horizontalInput = Input.GetAxisRaw("Horizontal");
    verticalInput = Input.GetAxisRaw("Vertical");
    moveDirection = PlayerOrientation.forward * verticalInput + PlayerOrientation.right * horizontalInput;

    if (grounded && moveDirection != Vector3.zero)
    {
        if (Input.GetKey(KeyCode.LeftShift))
        {
            anim.SetBool("Run", true);
            anim.SetBool("Walk", false);
            rb.AddForce(moveDirection.normalized * runspeed * 10f, ForceMode.Force);
        }
        else
        {
            anim.SetBool("Walk", true);
            anim.SetBool("Run", false);
            rb.AddForce(moveDirection.normalized * walkspeed * 10f, ForceMode.Force);
        }
    }
    else
    {
        anim.SetBool("Run", false);
        anim.SetBool("Walk", false);
    }
}

```

11. Membuat Transisi dan Memprogram Logika Jump.

Selanjutnya adalah membuat transisi Jump, cara pembuatannya sama seperti langkah sebelumnya, namun terdapat perbedaan state awal jump. Logika yang akan dilakukan adalah, animasi Jump bisa dilakukan pada state apapun, oleh karena itu state awal yang akan menghubungkan adalah **Any State**. Lalu hubungkan State jump dengan Idle. (jangan lupa berikan Kondisi di 2 Transisi yang baru di buat).



Uncheck **has Exit time** dan **Can Transform To Self**. Pasang **Motion/Animasi** Jump dari File **“Rifle Jump”**, Setelah itu lakukan penyesuaian logika pada Script **MovementLogic** sebagai berikut.

```
1 reference
private void Jump()
{
    if (Input.GetKeyDown(KeyCode.Space) && grounded)
    {
        rb.velocity = new Vector3(rb.velocity.x, 0f, rb.velocity.z);
        rb.AddForce(transform.up * jumppower, ForceMode.Impulse);
        grounded = false;
        anim.SetBool("Jump", true);
    }
    else if (!grounded)
    {
        rb.AddForce(Vector3.down * fallspeed * rb.mass, ForceMode.Force);
        if (aerialboost)
        {
            rb.AddForce(moveDirection.normalized * walkspeed * 10f * airMultiplier, ForceMode.Impulse);
            aerialboost = false;
        }
    }
}

1 reference
public void groundedchanger()
{
    grounded = true;
    aerialboost = true;
    anim.SetBool("Jump", false);
}
```

Jika sudah, Save Script kembali ke unity, dan masuk kedalam **Play Mode**.
Jika karakter memainkan animasi loncat ketika menekan tombol **Space**, Maka Konfigurasi dan Script sudah benar. jika masih belum, lakukan pengecekan ulang!

12. Selesai Untuk Materi Praktikum

Praktikum Dinyatakan Selesai Jika karakter sudah dapat memainkan animasi Walk, Run, dan Jump dengan logika animasi yang tepat.

Tugas Modul 3

Add Other Animation & Related Logic

1. Membuat Transisi dan pemrograman Animasi Idle Aim.

Buat **State**, **Transisi**, dan **kondisi** baru khusus digunakan untuk **Idle Aim**. Langkah langkahnya sama seperti Materi Praktikum. Jangan Lupa memasang **Motion/Animasi**, lalu program logikanya menggunakan script dibawah ini.

```
0 references
public void AimModeAdjuster()
{
    if (Input.GetKeyDown(KeyCode.Mouse1))
    {
        if (AimMode)
        {
            TPSMode = true;
            AimMode = false;
            anim.SetBool("AimMode", false);
        }
        else if (TPSMode)
        {
            TPSMode = false;
            AimMode = true;
            anim.SetBool("AimMode", true);
        }
    }
}
```

Ketika ditekan **RightClick**, Jika animasi berganti dari **Idle** ke **AimIdle**, dan Ketika di **RightClick** lagi animasi berpindah dari **AimIdle** ke **Idle**, maka Konfigurasi dan Script kalian sudah benar.

2. Membuat Transisi dan pemrograman Animasi Idle Shoot dan Walk Shoot.

Buat **State**, **Transisi**, dan **kondisi** baru khusus digunakan untuk **Idle Shoot** dan **Walk Shoot**. Langkah langkahnya sama seperti Materi Praktikum. Jangan Lupa memasang **Motion/Animasi**, lalu program logikanya menggunakan script dibawah ini.

```

1 reference
private void ShootLogic()
{
    if (Input.GetKey(KeyCode.Mouse0))
    {
        if (moveDirection.normalized != Vector3.zero)
        {
            anim.SetBool("WalkShoot", true);
            anim.SetBool("IdleShoot", false);
        }
        else
        {
            anim.SetBool("IdleShoot", true);
            anim.SetBool("WalkShoot", false);
        }
    }
    else
    {
        anim.SetBool("WalkShoot", false);
        anim.SetBool("IdleShoot", false);
    }
}

```

Jika Karakter dapat memainkan animasi **IdleShoot Ketika tidak bergerak**, dan memainkan **Walkshoot Ketika bergerak**, maka konfigurasi dan Script sudah benar.

3. Membuat Transisi dan pemrograman Animasi Death.

Buat **State, Transisi**, dan **kondisi** baru khusus digunakan untuk animasi **Death**. Langkah langkahnya sama seperti Materi Praktikum. Jangan Lupa memasang **Motion/Animasi**, lalu program logikanya menggunakan script dibawah ini. Untuk mentrigger Player mengurangi Healthnya, sementara menggunakan tombol **F**.

Unity Message | 0 references

```
void Update()
{
    Movement();
    Jump();
    ShootLogic();

    if (Input.GetKey(KeyCode.F))
    {
        PlayerGetHit(100f);
    }
}
```

2 references

```
public void PlayerGetHit(float damage)
{
    Debug.Log("Player Receive Damage - "+damage);
    HitPoints -= HitPoints - damage;

    if (HitPoints == 0f)
    {
        anim.SetBool("Death",true);
    }
}
```

jika Ketika ditekan tombol F karakter memainkan animasi Death, maka konfigurasi dan script sudah benar.

4. Selesai Untuk Tugas Modul 3

Jika Karakter sudah dapat menjalankan animasi Idle, Walk, Run, Jump, Idle Aim, Idle Shoot, Idle Walk, dan Death, maka Tugas Sudah selesai.